

codehive

Python: Deleting Files

Safe Removal and Directory Cleanup

1. The OS Module

Deleting files is a powerful action that interacts directly with the operating system. To do this, Python requires the `os` module.

The **`os`** module provides a portable way of using operating system dependent functionality, like removing files or navigating folders.

2. Removing Files

The primary function for file deletion is `os.remove()`. You simply pass the path of the file you wish to destroy.

```
import os
os.remove("old_report.txt")
```

Warning: This deletion is permanent. The file usually does not go to the Recycle Bin/Trash.

3. Defensive Programming

Attempting to remove a file that doesn't exist will crash your program with a `FileNotFoundError`.

Always use `os.path.exists()` as a safety check before executing a removal.

```
if os.path.exists("data.txt"):
    os.remove("data.txt")
else:
    print("Zero files found.")
```

4. Deleting Folders

To remove a directory (folder), Python provides the `os.rmdir()` function.

Constraint: `os.rmdir()` only works on **empty** folders. If the folder has even one file inside, it will throw an error.

5. Error Handling Patterns

Using `try...except` blocks is the most professional way to handle deletion, as it accounts for files currently in use by other programs.

```
try:
    os.remove("locked_file.txt")
except PermissionError:
    print("File is currently open elsewhere.")
except OSError as e:
    print(f"Error: {e.strerror}")
```

6. Deep Cleanup (shutil)

If you need to delete a folder and everything inside it, the standard `os` module isn't enough. You would typically use `shutil.rmtree()`.

`shutil` (Shell Utilities) is used for high-level file operations like copying and entire-directory removal.

7. Path Safety

Be extremely careful with relative paths when deleting. A script running in the wrong directory could delete the wrong "config.txt".

Pro Tip: Use `os.path.abspath()` to print the full path to the console before deleting, so you can verify the target during testing.

8. Temporary Files

Deletion is often used to clean up "Temp" files created during execution. Python's `tempfile` module can handle this automatically, but manually removing logs at the end of a script is standard practice.

Codehive

9. Knowledge Check

Exercise

Which function from the os module removes a file?

☐ os.delete()

☐ os.drop()

☒ **os.remove()**

10. Core Summary

- ✓ `os.remove()` for files.
- ✓ `os.rmdir()` for empty folders.
- ✓ `os.path.exists()` for safety.
- ✓ `try...except` for robust apps.
- ✓ "with great power comes great responsibility."

codehive

Cleanup & Management Series v1.0